1. Upload & Extract Dataset

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files dataset.zip
dataset.zip(application/x-zip-compressed) - 112266296 bytes, last modified: 3/21/2026 - 100% done
Saving dataset.zip to dataset.zip
```

```
import zipfile
import os

zip_path = "archive.zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("data")

print("Dataset extracted!")
```

```
Dataset extracted!
```

2. Check Dataset

```
os.listdir("data")
```

```
['test', 'train']
```

```
import os
print(os.listdir("data/train"))
```

```
['REAL', 'FAKE']
```

3. Data Preprocessing

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

img_size = (224, 224)
batch_size = 32

# Train data (with validation split)
train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_data = train_datagen.flow_from_directory(
    "data/train",
    target_size=img_size,
    batch_size=batch_size,
    class_mode='binary',
    subset='training'
)

val_data = train_datagen.flow_from_directory(
```

```
    "data/train",
    target_size=img_size,
    batch_size=batch_size,
    class_mode='binary',
    subset='validation'
)

# Test data (NO split)
test_datagen = ImageDataGenerator(rescale=1./255)

test_data = test_datagen.flow_from_directory(
    "data/test",
    target_size=img_size,
    batch_size=batch_size,
    class_mode='binary',
    shuffle=False
)
```

```
Found 80000 images belonging to 2 classes.
Found 20000 images belonging to 2 classes.
Found 20000 images belonging to 2 classes.
```

```
!pip install tensorflow keras matplotlib
```

```
~/local/lib/python3.12/dist-packages (2.19.0)
al/lib/python3.12/dist-packages (3.13.2)
~/local/lib/python3.12/dist-packages (3.10.0)
 /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
/local/lib/python3.12/dist-packages (from tensorflow) (26.0)
!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.12/dist-p
.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
~/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
sr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.78.0)
.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
l/lib/python3.12/dist-packages (from keras) (13.9.4)
al/lib/python3.12/dist-packages (from keras) (0.1.0)
cal/lib/python3.12/dist-packages (from keras) (0.19.0)
in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
9 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.46.3)
~<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.6)
usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
l in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)
7 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2026.2.25)
1 /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.10.2)
server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorfl
```

```
ı /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow) (3.1.6)
.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras) (4.0.0)
2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras) (2.19.2)
·/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)
 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=2.19.0->tensorflow)
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
```

4. Build CNN Model

```
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarni
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

5. Compile Model

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

6. Train Model

```
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=15
)
```

```
Epoch 1/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 156s 62ms/step - accuracy: 0.9862 - loss: 0.0380 - val_accuracy: 0.
Epoch 2/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 159s 63ms/step - accuracy: 0.9892 - loss: 0.0317 - val_accuracy: 0.
Epoch 3/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 154s 62ms/step - accuracy: 0.9913 - loss: 0.0260 - val_accuracy: 0.
Epoch 4/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 155s 62ms/step - accuracy: 0.9921 - loss: 0.0225 - val_accuracy: 0.
Epoch 5/15
```

```
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 155s 62ms/step - accuracy: 0.9932 - loss: 0.0206 - val_accuracy: 0.
Epoch 6/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 157s 63ms/step - accuracy: 0.9942 - loss: 0.0188 - val_accuracy: 0.
Epoch 7/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 156s 62ms/step - accuracy: 0.9946 - loss: 0.0174 - val_accuracy: 0.
Epoch 8/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 155s 62ms/step - accuracy: 0.9946 - loss: 0.0179 - val_accuracy: 0.
Epoch 9/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 156s 63ms/step - accuracy: 0.9954 - loss: 0.0131 - val_accuracy: 0.
Epoch 10/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 155s 62ms/step - accuracy: 0.9960 - loss: 0.0141 - val_accuracy: 0.
Epoch 11/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 154s 61ms/step - accuracy: 0.9965 - loss: 0.0118 - val_accuracy: 0.
Epoch 12/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 153s 61ms/step - accuracy: 0.9961 - loss: 0.0134 - val_accuracy: 0.
Epoch 13/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 155s 62ms/step - accuracy: 0.9966 - loss: 0.0124 - val_accuracy: 0.
Epoch 14/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 153s 61ms/step - accuracy: 0.9966 - loss: 0.0109 - val_accuracy: 0.
Epoch 15/15
2500/2500 ━━━━━━━━━━━━━━━━━━━━ 153s 61ms/step - accuracy: 0.9966 - loss: 0.0107 - val_accuracy: 0.
```

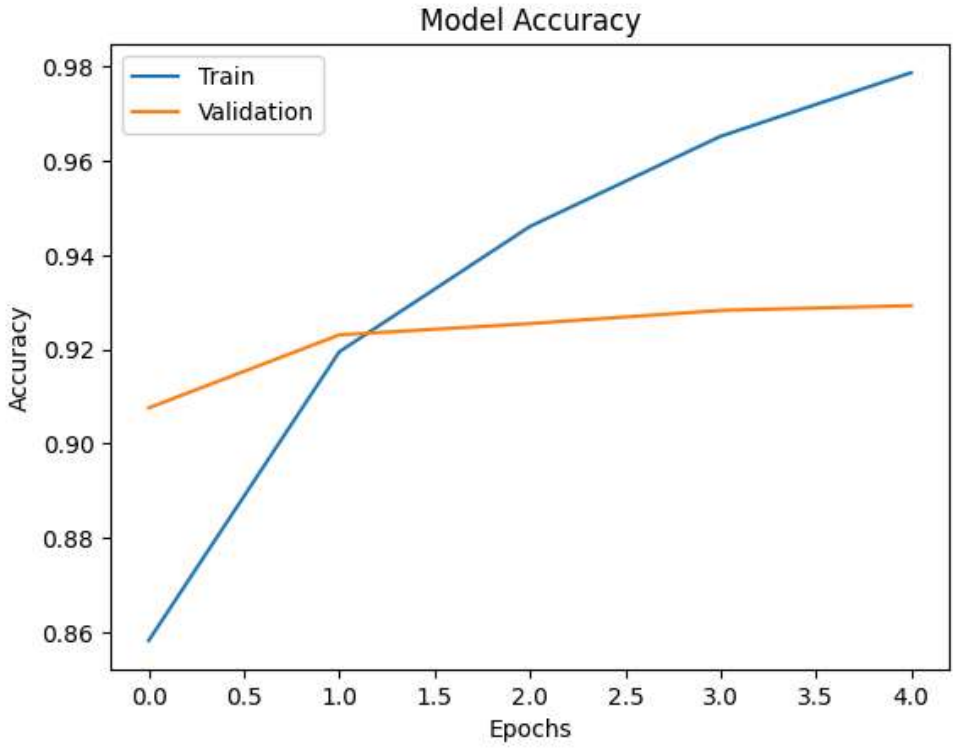
7. Evaluate Model

```
test_loss, test_acc = model.evaluate(test_data)
print("Test Accuracy:", test_acc)
```

```
625/625 ━━━━━━━━━━━━━━━━━━━━ 25s 40ms/step - accuracy: 0.9292 - loss: 0.2414
Test Accuracy: 0.9291999936103821
```

8. Plot Accuracy

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(['Train', 'Validation'])
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.show()
```



9. Class Mapping

```
print(train_data.class_indices)
```

```
{'FAKE': 0, 'REAL': 1}
```

```
prediction = model.predict(img_array)
print("Prediction value:", prediction)

if prediction[0][0] > 0.5:
    print("Real Image")
else:
    print("Fake Image")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Prediction value: [[0.00337731]]
Fake Image
```

```
print(prediction)
```

```
[[0.99874145]]
```

10. Upload & Predict Image

```
from tensorflow.keras.preprocessing import image
import numpy as np
from google.colab import files # Import files module

# Original line: img_path = "test.jpg"  # upload a test image

# Upload the test image file
print("Please upload your test image (e.g., test.jpg):")
```

```
uploaded = files.upload()

if uploaded:
    # Get the name of the first uploaded file
    img_path = list(uploaded.keys())[0]
    print(f"Uploaded image: {img_path}")
else:
    print("No file uploaded. Please upload an image file to proceed.")
    # Exit or handle error gracefully if no file was uploaded
    # For now, if no file is uploaded, the next line will still likely error out.

img = image.load_img(img_path, target_size=(224,224))
img_array = image.img_to_array(img)/255.0
img_array = np.expand_dims(img_array, axis=0)

prediction = model.predict(img_array)

if prediction[0][0] > 0.5:
    print("Fake Image")
else:
    print("Real Image")
```

```
Please upload your test image (e.g., test.jpg):
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 2.jpg to 2.jpg
Uploaded image: 2.jpg
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
Fake Image
```

11. Save Model

```
model.save("ai_image_detector.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_mc
```

```
from google.colab import files
files.download("ai_image_detector.h5")
```

```
!zip -r dataset.zip data/
```

```
  adding: data/train/FAKE/4059.jpg (deflated 18%)
  adding: data/train/FAKE/3411 (6).jpg (deflated 17%)
  adding: data/train/FAKE/4258 (2).jpg (deflated 18%)
  adding: data/train/FAKE/2328 (8).jpg (deflated 18%)
  adding: data/train/FAKE/2116.jpg (deflated 20%)
  adding: data/train/FAKE/4352 (10).jpg (deflated 19%)
  adding: data/train/FAKE/3849 (2).jpg (deflated 17%)
  adding: data/train/FAKE/1669 (5).jpg (deflated 18%)
  adding: data/train/FAKE/5402 (4).jpg (deflated 17%)
  adding: data/train/FAKE/2790.jpg (deflated 18%)
  adding: data/train/FAKE/3086 (2).jpg (deflated 18%)
  adding: data/train/FAKE/5575 (10).jpg (deflated 18%)
  adding: data/train/FAKE/3759 (4).jpg (deflated 19%)
  adding: data/train/FAKE/2156 (9).jpg (deflated 19%)
  adding: data/train/FAKE/2253 (5).jpg (deflated 18%)
  adding: data/train/FAKE/4785 (10).jpg (deflated 18%)
  adding: data/train/FAKE/1489 (2).jpg (deflated 19%)
  adding: data/train/FAKE/4742 (3).jpg (deflated 21%)
  adding: data/train/FAKE/5058 (2).jpg (deflated 18%)
  adding: data/train/FAKE/3787 (2).jpg (deflated 19%)
  adding: data/train/FAKE/2383 (7).jpg (deflated 16%)
  adding: data/train/FAKE/4310 (4).jpg (deflated 18%)
  adding: data/train/FAKE/4755 (3).jpg (deflated 18%)
  adding: data/train/FAKE/5646 (10).jpg (deflated 18%)
  adding: data/train/FAKE/3373 (5).jpg (deflated 20%)
  adding: data/train/FAKE/4240 (4).jpg (deflated 19%)
  adding: data/train/FAKE/1597 (2).jpg (deflated 18%)
  adding: data/train/FAKE/4241 (2).jpg (deflated 17%)
  adding: data/train/FAKE/3517 (8).jpg (deflated 17%)
  adding: data/train/FAKE/5721.jpg (deflated 18%)
  adding: data/train/FAKE/5929 (7).jpg (deflated 20%)
  adding: data/train/FAKE/5795 (6).jpg (deflated 18%)
  adding: data/train/FAKE/1801 (2).jpg (deflated 18%)
  adding: data/train/FAKE/4429 (6).jpg (deflated 18%)
  adding: data/train/FAKE/2049 (6).jpg (deflated 17%)
  adding: data/train/FAKE/5024 (3).jpg (deflated 19%)
  adding: data/train/FAKE/4014 (10).jpg (deflated 16%)
  adding: data/train/FAKE/3763 (3).jpg (deflated 16%)
  adding: data/train/FAKE/1435 (3).jpg (deflated 17%)
  adding: data/train/FAKE/2405 (5).jpg (deflated 15%)
  adding: data/train/FAKE/2966 (7).jpg (deflated 19%)
```

```
from google.colab import files
files.download("dataset.zip")
```

```
import pandas as pd

df = pd.DataFrame(history.history)
df.to_csv("results.csv")
```

```
files.download("results.csv")
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```
cnn = Sequential()
```

```
import zipfile

with zipfile.ZipFile('dataset.zip', 'r') as zip_ref:
    zip_ref.extractall('data')
```

```
import os
print(os.listdir('data'))
```

```
['data']
```

```
# =========================
# 1. IMPORTS
# =========================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report

# =========================
# 2. CHECK DATASET PATH
# =========================
print("Inside data folder:", os.listdir('data'))
print("Inside nested folder:", os.listdir('data/data'))

# =========================
# 3. DATA GENERATORS
# =========================
datagen = ImageDataGenerator(rescale=1./255)

train_generator = datagen.flow_from_directory(
    'data/data/train',
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary',
    shuffle=False
)

validation_generator = datagen.flow_from_directory(
    'data/data/test',
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary',
    shuffle=False
)

# =========================
# 4. CNN MODEL
# =========================
cnn = Sequential()

cnn.add(Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)))
```

```
cnn.add(MaxPooling2D(pool_size=(2,2)))

cnn.add(Conv2D(64, (3,3), activation='relu'))
cnn.add(MaxPooling2D(pool_size=(2,2)))

cnn.add(Flatten())
cnn.add(Dense(128, activation='relu'))
cnn.add(Dense(1, activation='sigmoid'))

cnn.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# =========================
# 5. TRAIN CNN (15 EPOCHS)
# =========================
history = cnn.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=15
)
# =========================
# 6. ACCURACY GRAPH
# =========================
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# =========================
# 7. FEATURE EXTRACTION
# =========================
train_generator.reset()
validation_generator.reset()

feature_model = Model(
    inputs=cnn.layers[0].input,
    outputs=cnn.layers[-2].output
)

X_train_features = feature_model.predict(train_generator, verbose=1)
X_test_features = feature_model.predict(validation_generator, verbose=1)

y_train = train_generator.classes
y_test = validation_generator.classes

# Flatten features
X_train_features = X_train_features.reshape(X_train_features.shape[0], -1)
X_test_features = X_test_features.reshape(X_test_features.shape[0], -1)

# =========================
# 8. ML MODELS
# =========================
svm_model = SVC(kernel='linear')
rf_model = RandomForestClassifier(n_estimators=100)
knn_model = KNeighborsClassifier(n_neighbors=5)

svm_model.fit(X_train_features, y_train)
rf_model.fit(X_train_features, y_train)
knn_model.fit(X_train_features, y_train)
```

```
knn_model.fit(X_train_features, y_train)

# ========================
# 9. PREDICTIONS
# ========================
svm_pred = svm_model.predict(X_test_features)
rf_pred = rf_model.predict(X_test_features)
knn_pred = knn_model.predict(X_test_features)

# ========================
# 10. CNN EVALUATION
# ========================
cnn_loss, cnn_acc = cnn.evaluate(validation_generator)

cnn_pred = (cnn.predict(validation_generator) > 0.5).astype("int32")

# ========================
# 11. RESULTS TABLE
# ========================
def evaluate(name, y_true, y_pred):
    return [
        name,
        accuracy_score(y_true, y_pred),
        precision_score(y_true, y_pred, average='weighted'),
        recall_score(y_true, y_pred, average='weighted')
    ]

results = []
results.append(["CNN", cnn_acc, "-", "-"])
results.append(evaluate("SVM", y_test, svm_pred))
results.append(evaluate("Random Forest", y_test, rf_pred))
results.append(evaluate("KNN", y_test, knn_pred))

df = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall"])

print("\n📊 Final Comparison:\n")
print(df)

# ========================
# 12. CNN CLASSIFICATION REPORT
# ========================
print("\n📄 CNN Classification Report:\n")
print(classification_report(y_test, cnn_pred))

# ========================
# 13. GRAPH (MODEL COMPARISON)
# ========================
plt.figure()
plt.bar(df["Model"], df["Accuracy"])
plt.title("Model Comparison")
plt.xlabel("Algorithms")
plt.ylabel("Accuracy")
plt.show()
```

```
Inside data folder: ['data']
Inside nested folder: ['train', 'test']
Found 100000 images belonging to 2 classes.
Found 20000 images belonging to 2 classes.
Epoch 1/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 173s 55ms/step - accuracy: 0.8033 - loss: 0.5352 - val_accuracy: 0.
Epoch 2/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 176s 56ms/step - accuracy: 0.8739 - loss: 0.2974 - val_accuracy: 0.
Epoch 3/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 165s 53ms/step - accuracy: 0.8933 - loss: 0.2598 - val_accuracy: 0.
Epoch 4/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 161s 51ms/step - accuracy: 0.9139 - loss: 0.2101 - val_accuracy: 0.
Epoch 5/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 161s 52ms/step - accuracy: 0.9291 - loss: 0.1738 - val_accuracy: 0.
Epoch 6/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 165s 53ms/step - accuracy: 0.9440 - loss: 0.1396 - val_accuracy: 0.
Epoch 7/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 162s 52ms/step - accuracy: 0.9559 - loss: 0.1097 - val_accuracy: 0.
Epoch 8/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 161s 52ms/step - accuracy: 0.9657 - loss: 0.0866 - val_accuracy: 0.
Epoch 9/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 160s 51ms/step - accuracy: 0.9718 - loss: 0.0716 - val_accuracy: 0.
Epoch 10/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 162s 52ms/step - accuracy: 0.9760 - loss: 0.0596 - val_accuracy: 0.
Epoch 11/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 161s 51ms/step - accuracy: 0.9808 - loss: 0.0495 - val_accuracy: 0.
Epoch 12/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 162s 52ms/step - accuracy: 0.9830 - loss: 0.0435 - val_accuracy: 0.
Epoch 13/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 162s 52ms/step - accuracy: 0.9848 - loss: 0.0396 - val_accuracy: 0.
Epoch 14/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 164s 52ms/step - accuracy: 0.9867 - loss: 0.0336 - val_accuracy: 0.
Epoch 15/15
3125/3125 ━━━━━━━━━━━━━━━━━━━━ 161s 51ms/step - accuracy: 0.9879 - loss: 0.0314 - val_accuracy: 0.
```

